

Reviewer Pack — Algebraic K-theory Rank and Circuit Entanglement Correlation

Entry 39d8c2bee6d2 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-31 10:57:35 UTC

Algebraic K-theory Rank and Circuit Entanglement Correlation

Entry ID: 39d8c2bee6d2 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-31 10:57:35 UTC

1. Conjecture

Field A (mathematical branch): Algebraic K-theory **Field B** (complexity object): Boolean circuit entanglement complexity

Statement:

For every d -regular graph G , the algebraic K-theory rank of its associated matrix representation is linearly correlated with its boolean circuit entanglement complexity, such that $\alpha(K(G)) = \Theta(\varepsilon^{-1} E(G))$, where ε is a constant and $E(G)$ is the minimum number of entangled gates required for a universal gate in the graph.

Rationale (proposer's reasoning):

Algebraic K-theory provides an invariant for studying the structure of rings and modules. By applying it to circuit representations, we may uncover new insights into the complexity of circuits, specifically their entanglement. This conjecture aims to bridge the gap between algebraic topology and quantum computation by correlating a complex algebraic invariant with a physical complexity measure.

Taxonomy category: ALKTHEORY_ENTANGLEMENT (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 0f9962651f90006f

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For each d -regular graph, if the algebraic K-theory rank $\alpha(K(G))$ is within a factor of $\varepsilon^{-1} E(G)$ with an absolute error less than or equal to 3 for at least 80% of the graphs tested, it supports the conjecture.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 5 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "algebraic K-theory" AND "boolean circuit entanglement complexity" - "matrix representation" IN "d-regular graph" AND "entanglement gates" - " $\alpha(K(G)) = \theta(\epsilon^{-1})E(G)$ "

Top relevant hits considered: - [http://arxiv.org/abs/math/0510146v2] Matrix Representation of Operators Using Frames - [http://arxiv.org/abs/2312.00361v1] Matrix representations of linear transformations on bicomplex space - [http://arxiv.org/abs/physics/0701105v3] Matrix Representation of Special Relativity - [s2:421b950135e527677096c93cabbcca7eb41fb71] A precise measurement of the direct CP violation parameter $\text{Re}(\epsilon(\text{cid:1})/\epsilon)$ - [s2:2004.06469] Efficient approximation algorithms for adaptive influence maximization

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.2s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(M):
        n = len(M)
        for col in range(n):
            pivot_row = None
            for row in range(col, n):
                if M[row][col] != 0:
                    pivot_row = row
                    break
            if pivot_row is None:
                continue
            M[pivot_row], M[col] = M[col], M[pivot_row]
            factor = Fraction(1, M[col][col])
            for r in range(n):
                if r == col:
                    continue
                multiplier = -M[r][col] * factor
                for c in range(n):
                    M[r][c] += multiplier * M[col][c]
        rank = sum(1 for row in M if any(M[row][col] != 0 for col in range(n)))
        return rank
```

```

def boolean_circuit_entanglement_complexity(G):
    # Placeholder function to compute the entanglement complexity
    # This is a dummy implementation and should be replaced with actual logic
    n = len(G)
    return random.randint(1, n) # Randomly generate a number between 1 and n

def d_regular_graph(n, d):
    G = [[] for _ in range(n)]
    edges_added = set()
    while len(edges_added) < (n * d) // 2:
        u = random.randint(0, n - 1)
        v = random.randint(0, n - 1)
        if u != v and (u, v) not in edges_added and (v, u) not in edges_added:
            G[u].append(v)
            G[v].append(u)
            edges_added.add((u, v))
    return G

def matrix_representation(G):
    n = len(G)
    M = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(i + 1, n):
            if j in G[i]:
                M[i][j] = 1
                M[j][i] = 1
    return M

def algebraic_k_theory_rank(M):
    rank = gaussian_elimination(M)
    return rank

n_max = 40
instances_tested = 30
metric_values = []

for _ in range(instances_tested):
    d = random.randint(2, 5) # Randomly generate a degree between 2 and 5
    G = d_regular_graph(n_max, d)
    M = matrix_representation(G)
    k_theory_rank = algebraic_k_theory_rank(M)
    entanglement_complexity = boolean_circuit_entanglement_complexity(G)

    if entanglement_complexity == 0:
        continue

    metric_values.append(k_theory_rank / entanglement_complexity)

if not metric_values:
    return {
        "metric_name": "algebraic_k_theory_rank_to_entanglement_complexity",
        "metric_value": None,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": False,
    }

```

```

        "counterexample": "mapping_undefined"
    }

    mean_C = sum(metric_values) / len(metric_values)
    std_C = math.sqrt(sum((x - mean_C) ** 2 for x in metric_values) / len(metric_values))
    support_fraction = sum(1 for val in metric_values if abs(val - mean_C) <= 3) / len(metric_valu

    return {
        "metric_name": "algebraic_k_theory_rank_to_entanglement_complexity",
        "metric_value": mean_C,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": support_fraction >= 0.8,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43,

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_C = sum(res["metric_value"] for res in results if res["metric_value"] is not None) / len
    std_C = math.sqrt(sum((res["metric_value"] - mean_C) ** 2 for res in results if res["metric_v
    support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

    if all(res["conjecture_holds"] for res in results):
        print(f"RESULT: SUPPORTED mean={mean_C} std={std_C} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_C} std={std_C} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(res["seed"] for res in results if not res["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_f

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_faa81458.py", line 119, in <module>
    result = run_trial(seed)
    ~~~~~^~~~~~

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_faa81458.py", line 82, in run_trial
    k_theory_rank = algebraic_k_theory_rank(M)
    ~~~~~^~~~~~

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_faa81458.py", line 71, in algebraic_k_t
    rank = gaussian_elimination(M)
    ~~~~~^~~~~~

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_faa81458.py", line 39, in gaussian_elim
    rank = sum(1 for row in M if any(M[row][col] != 0 for col in range(n)))
    ~~~~~
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_faa81458.py", line 39, in <genexpr>
    rank = sum(1 for row in M if any(M[row][col] != 0 for col in range(n)))
    ~~~~~
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_faa81458.py", line 39, in <genexpr>
    rank = sum(1 for row in M if any(M[row][col] != 0 for col in range(n)))
    ~~~~~

```

TypeError: list indices must be integers or slices, not list

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed during execution before producing any data, which prevents us from verifying the conjecture’s support or falsification. | next: Review and debug the test code to ensure it can run to completion without errors.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	21916
2	preregistration	ollama_remote	glm4:latest	0	0	9735
3	novelty	ollama_remote	glm4:latest	0	0	8663
4	novelty	ollama_remote	glm4:latest	0	0	22112
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16525
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12070
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16062
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20972
9	judge	ollama_remote	glm4:latest	0	0	9106

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 137162 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/39d8c2bee6d2.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/39d8c2bee6d2.jsonl for verification of provenance

4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/39d8c2bee6d2.tar.gz` (if generated)